



# ***Employee Payroll Management System using C***

**A Project Report**

*Submitted By*

KARTHIKEYAN ARUN KUMAR

SAHI BALA R

SRIRAM P

**PG Diploma in Embedded System Design & Development  
BATCH 90**

**INDIAN INSTITUTE OF EMBEDDED  
SYSTEMS**

KORAMANGALA, BENGALURU – 560095

**OCTOBER 2025**

## Project Name: Employee Payroll Management System using C

### Project Motive:

The primary motive of this project is to **simulate the core functionality of an employee payroll system**. This system handles the management of employee records, calculation of monthly salaries (including allowances and deductions), and persistence of data, providing a practical understanding of how basic record-keeping and financial management systems are designed and implemented.

### This Project Simulates:

- Employee Records: Storing and managing key details like ID, name, designation, and basic salary.
- Salary Calculation: Logic for calculating gross pay, standard deductions (5%), allowances (10%), and final net salary.
- Data Persistence: Using file handling (employees.txt) to store and retrieve employee data, ensuring records are maintained across program executions.
- User Interface: A simple, menu-driven interface for adding, viewing, and searching employee payroll records.

The project helps us learn backend logic, file handling, data structure design, and calculation implementation in a simplified form.

## Source Code (C Program)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define total 100

typedef struct {
    int id;
    char name[50];
    char designation[30];
    float initial;
    float allowances;
    float deductions;
    float final_;
} Employee;

Employee employees[total];
int Total_employees = 0;
const char *FILENAME = "employees.txt";

//---Function Declarations---

void menu();
void add();
void calculate(int index);
void show_all();
void search();
void save_file();
void load_file();

//---Main Function---

int main() {
    int choice;
    load_file();
    printf("---Welcome to the Employee Payroll System---\n");

    do {
        menu();
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            while(getchar() != '\n');
            choice = -1;
        }
        switch (choice) {
            case 1:
                add();
                save_file();
                break;
            case 2:
                show_all();
                break;
```

```

        case 3:
            search();
            break;
        case 4:
            printf("\nExiting the Program. Goodbye!\n");
            break;
        default:
            printf("\nInvalid choice. Please select an option between 1 and 4.\n");
    }
} while (choice != 4);

return 0;
}

//---Menu Function---

void menu() {
    printf("\n--- Main Menu ---\n");
    printf("1. Add New Employee\n");
    printf("2. View All Employees & Payroll\n");
    printf("3. Search Employee by ID\n");
    printf("4. Exit\n");
    printf("-----\n");
}

//---Calculating Allowance & Deduction---

void calculate(int i) {
    employees[i].allowances = employees[i].initial * 0.10; // 10% Allowance
    employees[i].deductions = employees[i].initial * 0.05; // 5% Deduction
    employees[i].final_ = employees[i].initial + employees[i].allowances - employees[i].deductions;
}

//---Adding A Employee---

void add() {
    if (Total_employees >= total) {
        printf("\nDatabase is full! Cannot add more employees.\n");
        return;
    }

    printf("\n--- Add New Employee ---\n");
    int newId;
    printf("Enter Employee ID: ");
    scanf("%d", &newId);
    while(getchar() != '\n');
    for (int i = 0; i < Total_employees; i++) {
        if (employees[i].id == newId) {
            printf("\nError: Employee with ID %d already exists.\n", newId);
            return;
        }
    }

    employees[Total_employees].id = newId;

```

```

printf("Enter Name: ");
fgets(employees[Total_employees].name, 50, stdin);
employees[Total_employees].name[strcspn(employees[Total_employees].name, "\n")] = 0;

printf("Enter Designation: ");
fgets(employees[Total_employees].designation, 30, stdin);
employees[Total_employees].designation[strcspn(employees[Total_employees].designation, "\n")] = 0;

printf("Enter Basic Salary: ");
scanf("%f", &employees[Total_employees].initial);
calculate(Total_employees);

printf("\nEmployee %s added successfully! Payroll calculated.\n", employees[Total_employees].name);
Total_employees++;
}

```

### //---Viewing The Added Employees List---

```

void show_all() {
    if (Total_employees == 0) {
        printf("\nNo employee records to display.\n");
        return;
    }

    printf("\n -----
\n");
    printf("| %-4s | %-20s | %-15s | %-15s | %-15s | %-15s | %-15s \n",
        "ID", "Name", "Designation", "Basic Salary", "Allowances", "Deductions", "Net Salary");
    printf(" -----
\n");

    for (int i = 0; i < Total_employees; i++) {
        printf("| %-4d | %-20s | %-15.2f | %-15.2f | %-15.2f | %-15.2f \n",
            employees[i].id,
            employees[i].name,
            employees[i].designation,
            employees[i].initial,
            employees[i].allowances,
            employees[i].deductions,
            employees[i].final_);
    }
    printf(" -----
\n");
}

void search() {
    if (Total_employees == 0) {
        printf("\nNo employee records to search.\n");
        return;
    }

    //---Search The Employee From List---

    int searchId;
    printf("\n--- Search Employee ---\n");

```

```

printf("Enter Employee ID to search: ");
if (scanf("%d", &searchId) != 1) {
    printf("\n Invalid input.\n");
    while(getchar() != '\n');
    return;
}

for (int i = 0; i < Total_employees; i++) {
    if (employees[i].id == searchId) {
        printf("\nEmployee Found:\n");
        printf(" ID: %d\n", employees[i].id);
        printf(" Name: %s\n", employees[i].name);
        printf(" Designation: %s\n", employees[i].designation);
        printf(" Basic Salary: %.2f\n", employees[i].initial);
        printf(" Net Salary: %.2f\n", employees[i].final_);
        return;
    }
}

printf("\nEmployee with ID %d not found.\n", searchId);
}

```

#### **//---Save The Added Details To A Text File(employees.txt)---**

```

void save_file() {
    FILE *file = fopen(FILENAME, "w");
    if (file == NULL) {
        printf("\nError opening file for saving!\n");
        return;
    }
    fprintf(file, "%d\n", Total_employees);
    for (int i = 0; i < Total_employees; i++) {
        fprintf(file, "%d,%s,%s,%.2f,%.2f,%.2f,%.2f\n",
            employees[i].id,
            employees[i].name,
            employees[i].designation,
            employees[i].initial,
            employees[i].allowances,
            employees[i].deductions,
            employees[i].final_);
    }
    fclose(file);
}

```

#### **//---Loading The Details Of Employees From File(employees.txt)---**

```

void load_file() {
    FILE *file = fopen(FILENAME, "r");
    if (file == NULL) {
        Total_employees = 0;
        return;
    }
    if (fscanf(file, "%d\n", &Total_employees) != 1) {
        Total_employees = 0;
        fclose(file);
    }
}

```

```

        return;
    }
    if (Total_employees > total) {
        printf("\nWarning: File has too many records. Reading maximum allowed.\n");
        Total_employees = total;
    }
    for (int i = 0; i < Total_employees; i++) {
        int result = fscanf(
            file,
            "%d,%49[^\n],%29[^\n],%f,%f,%f,%f\n",
            &employees[i].id,
            &employees[i].name,
            &employees[i].designation,
            &employees[i].initial,
            &employees[i].allowances,
            &employees[i].deductions,
            &employees[i].final_
        );
        if (result != 7) {
            printf("\nError reading record %d from file. Data might be corrupted.\n", i + 1);
            Total_employees = i;
            break;
        }
    }
    fclose(file);
}

```

## Demo Output

---Welcome to the Employee Payroll System---

--- Main Menu ---

1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit

-----

Enter your choice: 1

--- Add New Employee ---

Enter Employee ID: 101

Enter Name: Arun

Enter Designation: Senior Engineer

Enter Basic Salary: 50000

Employee Arun added successfully! Payroll calculated.

--- Main Menu ---

1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit

-----

Enter your choice: 1

--- Add New Employee ---

Enter Employee ID: 102

Enter Name: Bala

Enter Designation: Project Manager

Enter Basic Salary: 75000

Employee Bala added successfully! Payroll calculated.



--- Main Menu ---

1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit

-----

Enter your choice: 2

-----

-----

| ID         | Name       | Designation | Basic Salary | Allowances |  |
|------------|------------|-------------|--------------|------------|--|
| Deductions | Net Salary |             |              |            |  |

-----

-----

|          |          |                 |          |         |         |
|----------|----------|-----------------|----------|---------|---------|
| 101      | Arun     | Senior Engineer | 50000.00 | 5000.00 |         |
| 2500.00  | 52500.00 |                 |          |         |         |
| 102      | Bala     | Project Manager | 75000.00 | 7500.00 | 3750.00 |
| 78750.00 |          |                 |          |         |         |

-----

-----

--- Main Menu ---

1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID

4. Exit

-----

Enter your choice: 3

--- Search Employee ---

Enter Employee ID to search: 102

Employee Found:

ID: 102

Name: Bala

Designation: Project Manager

Basic Salary: 75000.00

Net Salary: 78750.00

--- Main Menu ---

1. Add New Employee

2. View All Employees & Payroll

3. Search Employee by ID

4. Exit

-----

Enter your choice: 4

Exiting the Program. Goodbye!

## Conclusion:

This project successfully simulates the working of a basic **Employee Payroll Management System**, covering:

- Employee Record Management: Using a structured approach to store employee data.
- Accurate Salary Calculation: Implementing defined business logic for calculating allowances (10% of Basic) and deductions (5% of Basic), leading to a net salary.
- Data Persistence: Utilizing robust file handling (employees.txt) to save and load data, a critical feature in real-world applications.
- User Interface: Providing a clear, menu-based system for interaction.

It provides a practical learning experience for students about how real-world record-keeping and financial management systems are structured and implemented using fundamental programming concepts.

## Output Screenshot:

```
---Welcome to the Employee Payroll System---

--- Main Menu ---
1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit
-----
Enter your choice: 1

--- Add New Employee ---
Enter Employee ID: 101
Enter Name: Arun
Enter Designation: Senior Engineer
Enter Basic Salary: 50000

Employee Arun added successfully! Payroll calculated.

--- Main Menu ---
1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit
-----
Enter your choice: 1

--- Add New Employee ---
Enter Employee ID: 102
Enter Name: Bala
Enter Designation: Project Manager
Enter Basic Salary: 75000

Employee Bala added successfully! Payroll calculated.

--- Main Menu ---
1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit
-----
Enter your choice: 2

-----
| ID | Name | Designation | Basic Salary | Allowances | Deductions | Net Salary |
-----
| 101 | Arun | Senior Engineer | 50000.00 | 5000.00 | 2500.00 | 52500.00 |
| 102 | Bala | Project Manager | 75000.00 | 7500.00 | 3750.00 | 78750.00 |
-----

--- Main Menu ---
1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit
-----
Enter your choice: 3

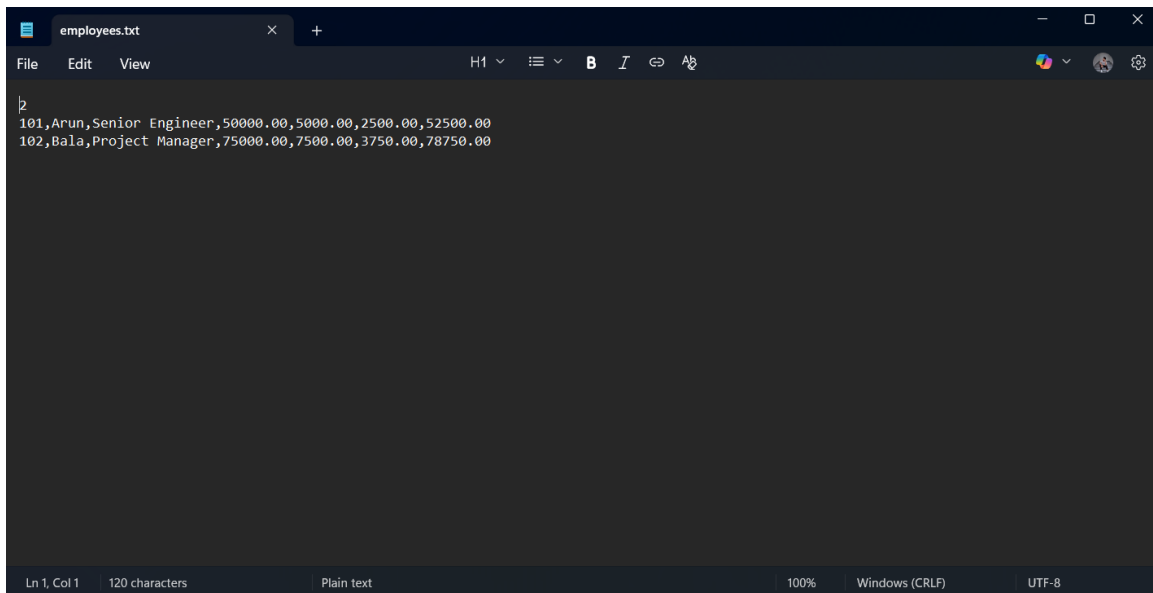
--- Search Employee ---
Enter Employee ID to search: 102

Employee Found:
ID: 102
Name: Bala
Designation: Project Manager
Basic Salary: 75000.00
Net Salary: 78750.00

--- Main Menu ---
1. Add New Employee
2. View All Employees & Payroll
3. Search Employee by ID
4. Exit
-----
Enter your choice: 4

Exiting the Program. Goodbye!
```

## employees.txt:



A screenshot of a code editor window titled 'employees.txt'. The editor has a dark theme and a menu bar with 'File', 'Edit', and 'View'. The toolbar includes icons for undo, redo, and search. The main text area contains two lines of CSV data. The status bar at the bottom shows 'Ln 1, Col 1', '120 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

```
2  
101,Arun,Senior Engineer,50000.00,5000.00,2500.00,52500.00  
102,Bala,Project Manager,75000.00,7500.00,3750.00,78750.00
```